

# DOMÍNIO MODERNO DE JAVASCRIPT

Algoritmos, Estruturas e Engenharia Web

ARQUITETURA PROFISSIONAL PARA APLICAÇÕES DE ALTO NÍVEL



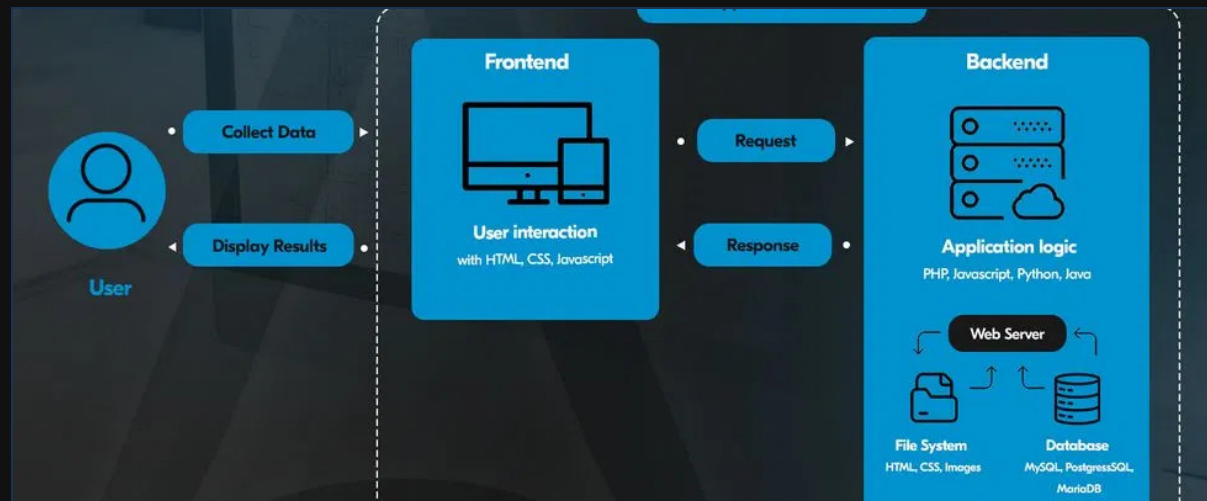
# A ENGENHARIA POR TRÁS DA WEB MODERNA

## ARQUITETURA DE 3 PILARES

**HTML:** Estrutura Semântica e Acessibilidade.

**CSS:** Design System, Layout e Experiência Visual.

**JavaScript:** Lógica de Negócio e Orquestração de Estado.



## O FLUXO MENTAL PROFISSIONAL

1. Usuário Interage
2. Evento Capturado
3. Resposta Lógica (Algoritmo)
4. Mutação do DOM
5. Atualização da Interface

*"A web moderna não é sobre documentos, mas sobre sistemas de estado sincronizados."*

# FUNDAMENTOS AVANÇADOS E LÓGICA PROFISSIONAL

## DOMÍNIO DO ESCOPO E MEMÓRIA

### Variáveis e Tipagem

Controle rigoroso de escopo léxico com `let` e `const`, priorizando a imutabilidade para evitar bugs de estado.

### Pensamento Algorítmico

Decomposição de problemas complexos em estruturas fundamentais: Sequência, Condição e Repetição.

### Engenharia de Lógica

A habilidade de traduzir requisitos de negócio em fluxos de dados determinísticos e eficientes.

## CLOSURES: O PODER DO ESCOPO

Funções que "lembram" do ambiente onde foram criadas, permitindo encapsulamento de dados privados.

```
function criarContador() {  
  let count = 0;  
  return () => ++count;  
}  
  
const counter = criarContador();  
console.log(counter()); // 1  
console.log(counter()); // 2
```

Essencial para módulos, fábricas de funções e padrões de design modernos.

# MANIPULAÇÃO ESTRATÉGICA DO DOM E BROWSER APIS



## DOM & EVENTOS

O DOM é uma API de árvore viva. A manipulação eficiente exige delegação de eventos e minimização de reflows.

```
el.addEventListener('click', e => {  
  e.target.classList.toggle('active');  
});
```



## PERSISTÊNCIA LOCAL

Gerenciamento de estado offline e cache de preferências do usuário sem dependência de servidor.

```
localStorage.setItem('theme', 'dark');  
const ui = localStorage.getItem('ui');
```



## ASSINCRONISMO

Orquestração de operações I/O não bloqueantes com Async/Await para interfaces fluidas.

```
async function fetchData() {  
  const res = await fetch(url);  
  return await res.json();  
}
```

# ESTRUTURAS DE DADOS: ORGANIZAÇÃO E MEMÓRIA

## ARRAYS

Acesso:  $O(1)$

Coleções ordenadas ideais para processamento sequencial. O uso de métodos funcionais (map, filter, reduce) garante código declarativo e limpo.

### CENÁRIO IDEAL

Listas de produtos, resultados de busca e buffers de dados ordenados.

## MAPS

Busca:  $O(1)$

Dicionários de alta performance. Diferente de objetos, aceitam qualquer tipo de chave e mantêm a ordem de inserção dos elementos.

### CENÁRIO IDEAL

Cache de requisições, contagem de frequências e dicionários dinâmicos.

## SETS

Unicidade:  $O(1)$

Coleções de valores únicos. Eliminam automaticamente duplicatas, otimizando a gestão de conjuntos de dados sem repetição.

### CENÁRIO IDEAL

Gestão de tags, filtragem de IDs únicos e controle de permissões de usuário.

# ALGORITMOS DE PERFORMANCE E CONTROLE DE FLUXO



## DEBOUNCE

Atrasa a execução de uma função até que um período de inatividade ocorra. Essencial para otimizar campos de busca e evitar chamadas excessivas de API.

### CASO DE USO

Input de busca: aguarda o usuário parar de digitar para disparar a requisição.



## THROTTLE

Garante que uma função seja executada no máximo uma vez a cada intervalo de tempo. Ideal para eventos de alta frequência como scroll e resize.

### CASO DE USO

Scroll infinito: limita a verificação da posição da página para economizar CPU.



## BUSCA BINÁRIA

Algoritmo de divisão e conquista para encontrar itens em listas ordenadas com eficiência logarítmica  $O(\log n)$ .

### VISUALIZAÇÃO



# ARQUITETURA E ENGENHARIA DE SOFTWARE COM JS

## Modularização

Uso de **ES Modules** para isolamento de responsabilidades. Facilita a manutenção, testes unitários e o reaproveitamento de lógica em diferentes partes do sistema.

import / export

Encapsulamento

Tree Shaking

## Estrutura de Projeto

Organização profissional de diretórios para garantir escalabilidade e clareza para a equipe.

```
src/  
├── assets/  
├── components/  
├── services/  
├── utils/  
└── main.js
```

## Padrões de Projeto

Soluções consolidadas para problemas recorrentes de design de software no ecossistema JavaScript.

Singleton

Observer

Factory

Module Pattern

*"Bons padrões reduzem a carga cognitiva e aumentam a previsibilidade do sistema."*

# PILHAS, FILAS E GERENCIAMENTO DE ESTADO

## STACK (PILHA)

LIFO: Last In, First Out

Gerencia a execução de funções e o escopo léxico. O motor JavaScript utiliza a Call Stack para rastrear onde o programa está.

- ✓ Histórico de navegação e botões "Voltar"
- ✓ Sistemas de Desfazer/Refazer (Undo/Redo)
- ✓ Recursividade e algoritmos de profundidade

## QUEUE (FILA)

FIFO: First In, First Out

Essencial para o processamento assíncrono. A Task Queue armazena callbacks aguardando o Event Loop.

- ✓ Fila de requisições HTTP e WebSockets
- ✓ Orquestração de animações sequenciais
- ✓ Processamento de mensagens em tempo real

---

*"O gerenciamento de estado é a arte de manter a 'verdade' da aplicação sincronizada através de fluxos de dados determinísticos."*

# ESTRUTURAS HIERÁRQUICAS E O ECOSISTEMA DOM

## Linked Lists

### Flexibilidade Dinâmica

Cadeias de nós onde cada elemento referencia o próximo. Essenciais para **inserções e remoções  $O(1)$**  sem a necessidade de reindexação total da memória.

*Base para a implementação de Buffers e sistemas de Undo/Redo complexos.*

## Árvores (DOM)

### Hierarquia Web

O navegador renderiza o HTML como uma **árvore de nós**. Manipular o DOM é, na essência, navegar e mutar uma estrutura hierárquica complexa.

*Algoritmos de busca em profundidade (DFS) e largura (BFS) são o segredo da renderização eficiente.*

## Hash Tables

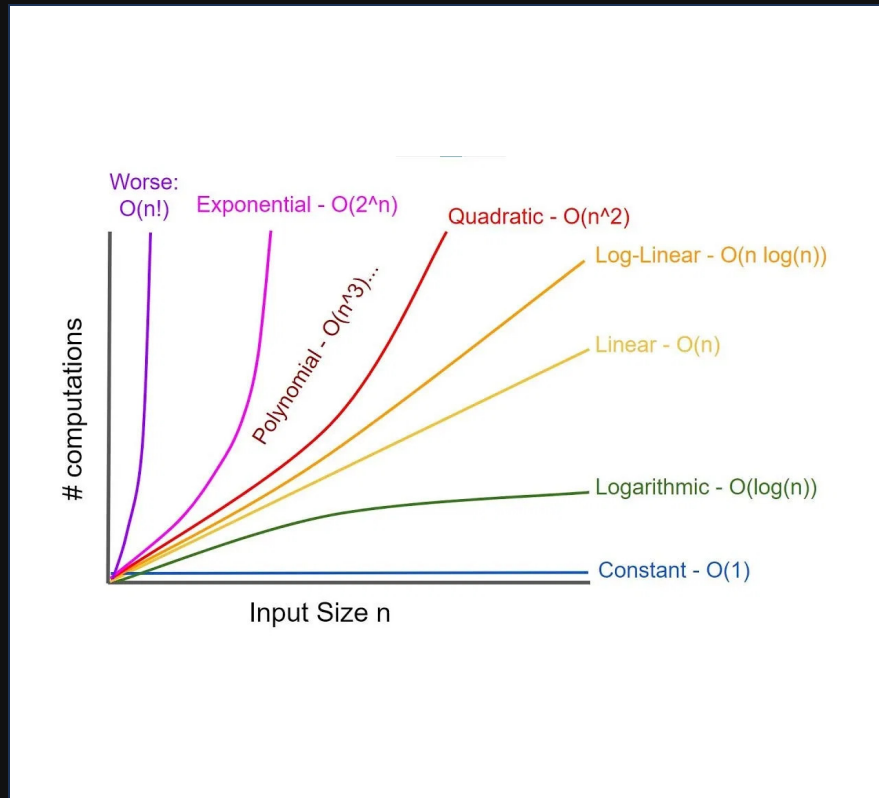
### Performance V8

O motor V8 utiliza Hash Tables internamente para garantir **acesso  $O(1)$**  às propriedades de Objetos e Maps, otimizando a busca por chaves.

*A eficiência do JavaScript moderno depende da implementação robusta de tabelas de dispersão.*

# ANÁLISE DE EFICIÊNCIA: A CIÊNCIA DO BIG O

## CURVA DE COMPLEXIDADE



A análise Big O descreve o limite superior do tempo de execução ou uso de memória conforme os dados crescem.

## COMPARATIVO DE OPERAÇÕES

| ESTRUTURA    | ACESSO      | BUSCA       | INSERÇÃO    |
|--------------|-------------|-------------|-------------|
| Array        | $O(1)$      | $O(n)$      | $O(1)^*$    |
| Map / Object | $O(1)$      | $O(1)$      | $O(1)$      |
| Set          | -           | $O(1)$      | $O(1)$      |
| Linked List  | $O(n)$      | $O(n)$      | $O(1)$      |
| Tree (DOM)   | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ |

\* Inserção no final do array. Inserções no início ou meio são  $O(n)$  devido ao deslocamento de índices.

**Trade-off:** Muitas vezes sacrificamos memória (espaço) para ganhar velocidade (tempo).

# TRILHA DE EVOLUÇÃO: DO INICIANTE AO PROFISSIONAL

## FASE 1

### Fundamentos Estáticos

#### FOCO

- > HTML5 Semântico
- > CSS3 & Flexbox
- > Responsividade

#### PROJETOS

- </> Landing Pages
- </> Portfólios

## FASE 2

### Lógica & Algoritmos

#### FOCO

- > Variáveis & Loops
- > Funções & Arrays
- > Objetos

#### PROJETOS

- </> Calculadoras
- </> Conversores

## FASE 3

### DOM & Browser APIs

#### FOCO

- > Event Listeners
- > LocalStorage
- > Manipulação DOM

#### PROJETOS

- </> Todo Apps
- </> Music Players

## FASE 4

### JS Moderno & APIs

#### FOCO

- > Async / Await
- > Fetch API
- > ES Modules

#### PROJETOS

- </> Dashboards
- </> API Viewers

# ENGENHARIA ALÉM DA SINTAXE

Aprender JavaScript não é apenas decorar comandos; é aprender a projetar soluções eficientes. A base sólida em algoritmos e estruturas de dados é o que separa o codificador do engenheiro de software profissional.

 **CONSTRUA O FUTURO COM EXCELÊNCIA TÉCNICA** 